Presenter Script: Quadruped Navigation using PPO and LiDAR in Unmapped Environments

---

Slide 1: Title Slide
"Good [morning/afternoon], everyone. I'm [Your Name], R&D Engineer, and today I'll be presenting the outcomes of a 4-month project where we tackled a complex robotics challenge: enabling a quadruped robot to navigate unmapped terrain using LiDAR and reinforcement learning, specifically Proximal Policy Optimization or PPO."

Slide 2: Agenda
"Here's our agenda for today. We'll begin by understanding the motivation behind this project. Then, we'll cover core RL concepts, zoom into PPO, and see how it applies to quadruped robots. We'll go over the system architecture, training, results, and end with future directions."

Slide 3: Introduction & Motivation
"Quadrupeds are ideal for rough and uneven terrain, where wheeled robots struggle. This makes them valuable in rescue operations, industrial inspection, and defense. However, most systems rely on maps or GPS. We aim to develop a robot that navigates completely mapless using only LiDAR, ensuring adaptability to any unknown environment."

Slide 4: Why Reinforcement Learning?
"Unlike rule-based systems, RL allows robots to learn optimal behavior through interaction. It uses rewards to guide learning. Among RL strategies, model-free methods work best when the environment is complex and unknown. PPO stands out due to its stability and efficiency, especially in continuous control like walking."

Slide 5: RL Fundamentals
"Let's review the RL loop. The agent is the robot. It perceives a state, like LiDAR scans and orientation. Based on a policy, it takes actions, like turning or moving forward. The environment provides rewards or penalties, and this loop repeats until the agent learns what actions lead to success."

Slide 6: PPO - Key Concepts
"PPO improves on earlier algorithms like TRPO. It restricts large, unstable updates by clipping the policy ratio, which maintains training stability. It uses two main loss functions: one for the actor (policy) and one for the critic (value estimation). We'll see the math soon."

Slide 7: PPO for Quadruped Navigation
"In our system, the robot's input is just LiDAR data and its current orientation. The PPO policy network takes this input and outputs discrete velocity commands. The robot learns to associate sensor readings with the best movement, without any map or human intervention."

Slide 8: Value Function (Critic)
"The critic estimates how good a state is. It helps stabilize learning by reducing the variance in advantage estimates. The value loss is calculated by comparing the predicted value to the actual reward received over time."

Slide 9: Our Exact Approach
"We define the robot's state using 5 elements: LiDAR data, goal direction, previous velocity, orientation, and angle to goal. The action space is discretized to simplify learning. We use a multilayer perceptron (MLP) for both actor and critic. Rewards are carefully designed to encourage goal-reaching and penalize collisions."

Slide 10: State Representation
"Mathematically, the state is: $s_t = \{x_t, p_t, v_{t-1}, \theta_t, \alpha_t\}$. Each of these plays a role in navigation. LiDAR detects obstacles, p_t guides the robot toward the target, and angles ensure it's facing the right way."

Slide 11: Preprocessing - Method 1
"We process 720 LiDAR points into 120 sectors. Orientation is one-hot encoded. This yields a high-dimensional input that provides rich environmental detail, but it's compute-intensive."

Slide 12: PPO Network Architecture
"The MLP has 3 hidden layers. After that, it splits into two heads: one for the actor, which outputs the action probabilities, and one for the critic, which predicts the state value. This shared structure improves learning efficiency."

Slide 13: Reward Function
"Here’s our reward function. It combines distance to goal, angular alignment, a bonus for reaching the goal, and penalties for collision. This balances exploration with safety and goal-directed movement."

Slide 14: Preprocessing - Method 2
"To optimize for embedded systems, we try Method 2: 30 LiDAR readings grouped into 10. We take the minimum in each, yielding a 10D input. This is faster, and surprisingly effective in simpler environments."

Slide 15: Preprocessing Comparison
"Method 1 provides higher precision, but is slower. Method 2 trades off some detail for speed, which is crucial for real-time navigation. We choose based on the deployment platform."

Slide 16: Training Strategy
"We train the robot in a simulated world with randomized obstacles and goal placements. PPO is updated using advantage estimation and clipping. We monitor success rate, collisions, and episode rewards."

Slide 17: Results & Observations
"Over time, reward increases and collision rate drops. The robot learns to take shorter, more efficient paths. Even in unseen environments, it performs well. PPO shows strong convergence and generalization."

Slide 18: Challenges
"Initial learning was hard due to sparse rewards. Encoding orientation correctly was critical. We had to carefully tune exploration parameters to avoid getting stuck. Also, real-world deployment is still in progress."

Slide 19: Future Work
"Next, we plan to test on real hardware, add camera inputs, and support moving obstacles. We also aim to improve robustness under noisy or partial sensor data."

Slide 20: Q&A
"Thank you! I'm happy to take any questions."

Appendix: Symbols, Acronyms, and Equations

Symbols:

- s_t: Current state
- x_t: LiDAR data
- p_t: Relative position to goal
- v_{t-1}: Previous velocity
- θ_t: Orientation (yaw angle)
- α_t: Angular offset to goal
- a_t: Action taken
- R_t: Cumulative reward
- $V(s)$: Value of state
- π: Policy

Acronyms:

- RL: Reinforcement Learning
- PPO: Proximal Policy Optimization
- MLP: Multilayer Perceptron
- LiDAR: Light Detection and Ranging
- DQN: Deep Q-Network

- TRPO: Trust Region Policy Optimization
- DDPG: Deep Deterministic Policy Gradient

Formulas:

1. **Policy Loss (Clipped Surrogate):**
   $LCLIP(θ)=E^t[min(rt(θ)A^t,clip(rt(θ),1−ϵ,1+ϵ)A^t)]L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)\right]$
2. **Value Loss (Critic):**
   $LV(θ)=12(Vθ(st)−Rt)2L^V(\theta) = \frac{1}{2} (V_\theta(s_t) - R_t)^2$
3. **State Vector:**
   $st={xt,pt,vt−1,θt,αt}s_t = \{x_t, p_t, v_{t-1}, \theta_t, \alpha_t\}$
4. **Reward Function:**
   $Rt=−α⋅dt−β⋅∣θt∣+γ⋅1goal−δ⋅1collisionR_t = -\alpha \cdot d_t - \beta \cdot |\theta_t| + \gamma \cdot \mathbb{1}_{\text{goal}} - \delta \cdot \mathbb{1}_{\text{collision}}$